

PAGE 2026 — {ospsuite} + {esqlabsR} Cheatsheet

Workshop reference — 2 pages

Pavel Balazki · Wilbert de Witte

2026-06-01

{ospsuite} essentials

Workflow

load → inspect → modify → run → extract → analyze
→ plot

Load + save

```
sim <- loadSimulation("path/Model.pkml")
saveSimulation(sim, "path/Model_v2.pkml")
# Independent copy? Reload
sim_copy <- loadSimulation("path/Model.pkml")
```

Inspect

```
getAllParameterPathsIn(sim)
getAllMoleculePathsIn(sim)
getAllContainerPathsIn(sim)
getAllParametersMatching("**|Volume", sim)
```

Parameter access

```
p <- getParameter("Organism|Liver|Volume", sim)
p$value; p$unit
setParameterValuesByPath("Organism|Weight", 75, sim, units =
  ↪ "kg")
```

Run

```
res <- runSimulations(sim)[[1]] # single
res_list <- runSimulations(list(sim1, sim2)) # multiple
```

Extract

```
out <- getOutputValues(res, quantitiesOrPaths =
  ↪ "...|Concentration")
out$data # → data.frame: Time, paths, values
```

Plot — DataCombined

```
dc <- DataCombined$new()
dc$addSimulationResults(res)
dc$addObservedData(loadDataSetFromPKML("obs.pkml"))
plotTimeProfile(dc) # time profile
plotPredictedVsObserved(dc) # GoF
plotResidualsVsCovariate(dc, xAxis = "time") #
  ↪ residuals
```

PK params

```
pk <- calculatePKAnalyses(res, "...|Concentration")
pkAnalysesAsDataFrame(pk) # AUC, C_max, t_max, t½
addUserDefinedPKParameter(...)
```

Local sensitivity

```
sa <- SensitivityAnalysis$new(
  simulation = sim,
  parameterPaths = c("Aciclovir|Lipophilicity",
    "Aciclovir|Fraction unbound (plasma,
      ↪ reference value)"))
sa$addParameterPaths("Organism|Liver|Volume")
sa$numberOfSteps <- 1 # default 2
sa$variationRange <- 0.2 # default 0.1 (symmetric)

saResult <- runSensitivityAnalysis(sa)
saResult$allPKParameterSensitivitiesFor(
  pkParameterName = "C_max",
  outputPath = saResult$allQuantityPaths[[1]],
  totalSensitivityThreshold = 0.9)

exportSensitivityAnalysisResultsToCSV(saResult, "mySA.csv")
```

Units

```
toUnit("Concentration (mass)", 1, "mg/L")
toBaseUnit("Volume", 70, "ml")
toDisplayUnit(p, p$value)
```

Path conventions

Organism|Liver|Volume

Levels: organ → compartment → parameter. | separates levels. ** = recursive wildcard.

Parameter identification

Optimization parameter

```
piPar <- PIParameters$new(
  parameters = getParameter("Aciclovir|Lipophilicity", sim))
piPar$minValue <- -5
piPar$maxValue <- 5
piPar$startValue <- 2 # optional
```

Observed data → output mapping

```
obs <- loadDataSetsFromExcel(
  "Data/Obs.xlsx", createImporterConfigurationForFile(
    filePath = "Data/Obs.xlsx"
  ))

simPath <- "Organism|PeripheralVenousBlood|Aciclovir|Plasma
  ↪ (Peripheral Venous Blood)"
mapping <- PIOutputMapping$new(
  quantity = getQuantity(simPath, container = sim))
mapping$addObservedDataSets(obs$`Aciclovir.Vergin1995.IV250`)
```

Configure + run

```

cfg <- PIConfiguration$new()
cfg$algorithm <- "DEoptim" # or "LevenbergMarquardt"
cfg$autoEstimateCI <- TRUE

piTask <- ParameterIdentification$new(
  simulations = sim,
  parameters = piPar,
  outputMappings = mapping,
  configuration = cfg)

res <- piTask$run()
piTask$plotResults()

```

{esqlabsR} essentials

Project structure

```

MyProject/
├── ProjectConfiguration.xlsx      ← master
├── Models/
│   ├── Simulations/             *.pkml
├── Configurations/
│   ├── Scenarios.xlsx
│   ├── ModelParameters.xlsx
│   ├── Applications.xlsx
│   ├── Individuals.xlsx
│   ├── Populations.xlsx
│   ├── PopulationsCSV/
│   ├── ParameterIdentification.xlsx
│   └── Plots.xlsx
├── Data/                        *.xlsx
├── Results/
└── Reports/                     *.qmd

```

Initialize

```

library(esqlabsR)
initProject(path = "~/MyProject")

```

Project object

```

projectConfig <- esqlabsR::createProjectConfiguration(
  ↪ "workshops/resources/projects/Midazolam-PAGE/ProjectConfiguration"
  ↪ )
print(projectConfig) # full summary

projectConfig$modelFolder #
  ↪ Models/Simulations
projectConfig$configurationsFolder # Configurations/
projectConfig$scenariosFile #
  ↪ Configurations/Scenarios.xlsx
projectConfig$individualsFile #
  ↪ Configurations/Individuals.xlsx
projectConfig$populationsFile #
  ↪ Configurations/Populations.xlsx
projectConfig$parameterIdentificationFile #
  ↪ Configurations/ParameterIdentification.xlsx
projectConfig$plotsFile #
  ↪ Configurations/Plots.xlsx
projectConfig$outputFolder # Results/
projectConfig$dataFile #
  ↪ Data/Midazolam_obs.xlsx

```

Load + run scenarios

```

cfg <- readScenarioConfigurationFromExcel(
  scenarioNames = c("PO_7.5mg", "IV_2mg"),
  projectConfiguration = project)
sc <- createScenarios(cfg)
res <- runScenarios(scenarios = sc)

```

Snapshot / restore

```

snapshotProjectConfiguration(project) # Excel → JSON
  ↪ snapshot
restoreProjectConfiguration(project) # JSON → Excel
projectConfigurationStatus(project) # diff

```

Plots from Excel

```
# parameters from YAML
params$dose_mg

# auto-run scenarios
cfg  <- readScenarioConfigurationFromExcel(params$scenario,
  ↪  project)
sc   <- createScenarios(cfg)
res  <- runScenarios(sc)
plots <- createPlotsFromExcel(plotGridNames = "MyPlot",
  simulatedScenarios = res,
  projectConfiguration = project)
```